

AURE Security Audit Report

Browser Extension Security Assessment

Assessment date: August 17, 2026

Target: AURE_extension source repository

Executive summary: The reviewed Manifest V3 extension did not show confirmed remote code execution, direct DOM XSS, or a webpage-to-background postMessage bridge in the current source. The strongest confirmed weaknesses are around authentication token storage, trust of plaintext localhost services, and limited authorization checks around privileged internal messaging.

Finding totals: 0 Critical, 0 High, 2 Medium, 2 Low, 0 Informational. Backend-side issues such as IDOR, BOLA, JWT validation, password-reset abuse, and rate limiting could not be fully verified because the backend source was not included in the provided repository.

Architecture and attack surface

- Manifest V3 extension with one service worker, one content script, popup UI, and side panel UI.
- Permissions: storage, activeTab, sidePanel, tabs.
- Host permissions include ChatGPT, Claude, Gemini, Grok, Perplexity, DeepSeek, Copilot, and localhost or 127.0.0.1 backend endpoints.
- Core trust boundaries: webpage DOM -> content script -> background worker -> localhost API -> user account data.

Vulnerability summary

AURE-01	Medium	Session tokens stored in extension-readable storage	Account takeover if extension context is compromised
AURE-02	Medium	Plain HTTP localhost auth and API trust	Credential capture, token theft, prompt and history exposure
AURE-03	Low	Background handlers lack sender-level authorization checks	Sensitive actions available to any compromised internal extension page
AURE-04	Low	Broader tab access and generic prompt injection targeting	Larger blast radius after compromise, weaker least-privilege posture

Detailed findings

AURE-01 - Session tokens are persisted in extension-readable storage

Severity: Medium | CWE-922, CWE-312

Location: src/stores/auth.store.ts and src/lib/cookies.ts

What is the problem: The login and session-loading paths copy access tokens into chrome.storage.local under promptiq_token and apiToken, then reuse them automatically in the API client. The cookie helper also falls back to document.cookie when available. This means the extension stores bearer credentials in places readable by extension JavaScript rather than relying solely on server-managed HttpOnly cookies.

Evidence: auth.store.ts lines 103-117, 169-182, and 243-247; cookies.ts lines 71-75 and 103-107; api/client.ts lines 49-55 and 95-97.

Impact: If any extension UI context is compromised in the future, the attacker can recover the bearer token and impersonate the user against profile, save, history, and prompt endpoints. This is a realistic account takeover and data exposure path after extension compromise.

Recommended fix: Keep session state only in Secure, HttpOnly, SameSite cookies where possible. Stop copying access tokens into chrome.storage.local. Remove document.cookie fallback for auth tokens, and either enforce the encryptData setting or remove it from the product.

AURE-02 - The extension trusts plaintext localhost services for auth and API traffic

Severity: Medium | CWE-319, CWE-346

Location: `src/api/client.ts`, `src/lib/cookies.ts`, and `src/components/auth/AuthView.tsx`

What is the problem: The default API endpoint is `http://127.0.0.1:8000/api/v1`, the cookie origin defaults to `http://127.0.0.1:8000`, and the Google sign-in button opens `http://localhost:3000/auth`. The extension will also honor a custom `advanced.apiEndpoint` value without enforcing HTTPS for non-loopback origins.

Evidence: `api/client.ts` lines 40-43 and 127-133; `cookies.ts` line 7 and lines 88-97; `AuthView.tsx` line 237.

Impact: A malicious or unexpected local process can impersonate the backend, capture credentials or tokens, and collect prompts or history over plaintext HTTP. If users point the extension at another insecure endpoint, Authorization and X-Current-User headers will be sent there as well.

Recommended fix: Require HTTPS for all non-loopback endpoints, explicitly allow only approved origins, and avoid plain HTTP for authentication flows. Treat localhost as a development-only mode with strong warnings and separate build-time controls.

AURE-03 - Privileged background handlers do not enforce sender-based authorization

Severity: Low | CWE-285, CWE-639

Location: `src/lib/messaging.ts` and `src/entrypoints/background/index.ts`

What is the problem: The message router dispatches requests by message type without checking `sender.url`, `sender.id`, or intended caller role. Sensitive handlers such as `GET_HISTORY`, `DELETE_PROMPT`, `UPDATE_SETTINGS`, `SAVE_VERSION`, and `FILL_PROMPT` assume any caller inside the extension is allowed to use them.

Evidence: `lib/messaging.ts` lines 91-114; `background/index.ts` lines 153-180 and 198-241.

Impact: I did not find a direct webpage-to-background bridge in the current source, so this is not a confirmed remote exploit from the open web. However, if any extension page is compromised later, sensitive background capabilities are immediately available without further authorization checks.

Recommended fix: Apply per-action authorization in the background worker. Validate sender context and limit high-risk actions to the specific popup, side panel, or trusted content-script origins that need them.

AURE-04 - Tabs permission and generic prompt-fill targeting are broader than necessary

Severity: Low | CWE-250

Location: `manifest.json` and `src/entrypoints/background/index.ts`

What is the problem: The extension requests tabs in addition to `activeTab` and uses `FILL_PROMPT` logic that enumerates all tabs and then chooses any active HTTP or HTTPS page before falling back to known AI domains. This is broader than the core product need suggested by the supported site list.

Evidence: manifest.json permissions block; background/index.ts lines 198-237.

Impact: The broader permission and loose targeting increase the blast radius if the extension is abused internally. It also weakens least-privilege posture by allowing prompt injection targeting logic to consider unrelated browser tabs.

Recommended fix: Prefer activeTab with strict host checks and refuse fill actions outside the supported AI platforms unless the user explicitly enables a controlled custom-site mode.

Attack path analysis

- Local malicious process -> binds localhost backend endpoint -> receives credentials or bearer token -> accesses user profile, prompts, and saved prompt history.
- Future compromise of popup or side panel code -> reads promptiq_token or apiToken from chrome.storage.local -> reuses bearer token through API client -> unauthorized access to user data.

Security test matrix

XSS and DOM injection	PASS	No confirmed exploit path from attacker-controlled input to dangerous DOM sink in current source
Message passing	PARTIAL	No external webpage bridge found, but internal background handlers lack sender-level checks
Token security	FAIL	Bearer tokens persisted in local storage and reused automatically
Transport security	FAIL	Plain HTTP localhost defaults used for auth and API
Extension permissions	PARTIAL	No dangerous web-accessible scripts, but tabs scope is broader than needed
Secrets scan	PASS	No hardcoded secrets found in repository review
Dependency audit	PARTIAL	npm audit could not be completed because the audit endpoint was unreachable in this environment on August 17, 2026
Backend authorization	NOT VERIFIED	Backend source not included, so IDOR or BOLA could not be confirmed from extension code alone

Positive security controls

- No externally_connectable configuration was present in the manifest.
- No unsafe eval, Function constructor use, or direct webpage postMessage bridge was identified in the current source.
- Content-script scope is limited to the declared AI platform domains rather than all sites.

Remediation roadmap

- P0: Remove token duplication into chrome.storage.local and avoid document.cookie auth fallback.
- P0: Enforce secure transport rules and block insecure custom API endpoints by default.
- P1: Add sender-aware authorization checks for privileged background actions.
- P1: Reduce permissions and tighten prompt-fill tab targeting logic.
- P2: Add explicit threat-model tests for extension trust boundaries and malicious-local-service scenarios.

Verification limits

This report is evidence-driven from the provided extension source code only. The backend implementation was not present, so server-side authentication, authorization, rate limiting, JWT handling, and cross-user data access could not be verified from source. The local environment also did not provide the usual PDF rendering toolchain, so final PDF QA was limited to structural checks rather than image-based page rendering.

End of report.